ACTIVIDAD

Dosier de Actividades del Tema: Almacenamiento de objetos en ficheros. Serialización

Desarrollo de Aplicaciones Web/ Desarrollo de Aplicaciones Multiplataforma/Administración de Sistemas Informáticos y Redes

Programación

Linkia FP

Se debe entregar un zip que contenga todos los archivos. java que has creado. Para poder aprobar un ejercicio, éste debe poder ejecutarse sin errores.

En esta actividad, se te pide que implementes la serialización y deserialización de objetos en Java.

EJERCICIO 1: Crea una clase Libro con los siguientes atributos:

- String autor, String titulo, int anyo, double precio.
- Crea los métodos getters & setters y el constructor.

EJERCICIO 2: Implementa la interfaz Serializable en la clase Libro para permitir la serialización de objetos de esta clase.

public class Libro implements Serializable { // esto es la cabecera de la clase.

EJERCICIO 3: Dentro del método main, completa el código necesario para realizar la serialización de un objeto Libro y guardarlo en un archivo llamado "libros". Asegúrate de seguir los pasos adecuados para abrir los flujos de salida y escribir el objeto en el archivo.

EJERCICIO 4: Completa el código para realizar la deserialización del objeto Libro desde el archivo "libros " y mostrar por pantalla los atributos del libro deserializado (título, autor y año). El atributo precio no se tiene que serializar.

EJERCICIO 5: Ejecuta el programa y verifica que la serialización y deserialización del objeto se realice correctamente. Asegúrate de que los atributos del libro deserializado sean los mismos que los del libro original.

Recuerda que el campo autor debe declararse como transient en la clase Libro para evitar que se serialice. También asegúrate de manejar correctamente las excepciones que puedan producirse durante la serialización y deserialización.

Nota personal:

Este ejercicio lo plantearé diferente en la documentación, en vez de ir poniendo bajo cada enunciado el codigo que le toca pondré solamente el main y la clase del tirón y añadiré mis explicaciones.

EJEMPLO DE RESULTADO

```
El objeto se ha serializado correctamente.
El objeto se ha deserializado correctamente.
Título: 1984
Autor: George Orwell
Año: 1949
BUILD SUCCESSFUL (total time: 0 seconds)
```

Clase Libro:

```
package ACT12;
 import java.io.Serializable;

/**
 *
 * @author cristiancantero
 */

public class Libro implements Serializable {

    private String autor;
    private String titulo;
    private int anyo;
    private transient double precio; // NO se serializa

    public Libro(String autor, String titulo, int anyo, double precio) {
        this.autor = autor;
```


Linkia FP

```
    this.titulo = titulo;
    this.anyo = anyo;
    this.precio = precio;
}

public String getAutor() {
    return autor;
}

public void setAutor(String autor) {
    this.autor = autor;
}

public String getTitulo() {
    return titulo;
}

public void setTitulo(String titulo) {
    this.titulo = titulo;
}

public int getAnyo() {
    return anyo;
}

public void setAnyo(int anyo) {
    this.anyo = anyo;
}

public double getPrecio() {
    return precio;
```

```
    }

    public void setPrecio(double precio) {
        this.precio = precio;
    }

    @Override
    public String toString() {
        return "Libro{" +
                "autor='" + autor + '\'' +
                ", titulo='" + titulo + '\'' +
                ", año=" + anyo +
                ", precio=" + precio +
                '}';
    }
}
```

Como se puede ver la interfaz Serializabe está implementada y el atributo precio no se serializa porque utilizo el modificador transient.

Ahora vamos a la clase main

```
package ACT12;
import java.io.*;
/**
 *
 * @author cristiancantero
 */


public class Main {

    public static void main(String[] args) {

        Libro libro = new Libro("George Orwell", "1984", 1949, 19.99);

        // SERIALIZAR
        try {
            ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("libros"));
            oos.writeObject(libro);
            oos.close();
            System.out.println("Libro guardado correctamente");
        } catch (IOException e) {
            e.printStackTrace();
        }

        // DESERIALIZAR
        try {
            ObjectInputStream ois = new ObjectInputStream(new FileInputStream("libros"));
            Libro libroLeido = (Libro) ois.readObject();
```

```
            ois.close();

            System.out.println("Libro recuperado:");
            System.out.println("Titulo: " + libroLeido.getTitulo());
            System.out.println("Autor: " + libroLeido.getAutor());
            System.out.println("Año: " + libroLeido.getAnyo());


        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

He decidido usar try/catch porque asi puedo controlar las excepciones. Como por ejemplo que el archivo no exista o que haya problemas al leer o escribir.

Dentro del catch he utilizado e.printStackTrace(). No es algo que haya usado antes, pero investigando he visto que sirve para mostrar por consola el error completo, incluyendo dónde se ha producido. Tampoco lo he puesto a prueba asi que realmente no se como funicona.

Se me ocurrió que cabia la posibilidad que quisieras que hiciese exceptions personalizadas ya que el tema anterior va de eso. Pero sinceramente, tengo que estudiar mucho HashMap, ArrayList, escritura/lectura secuencial y BDD. Y queda 1 mes y una semana para el examen.

He investigado un poquito el tema de la serialización y resulta que ya no se usa para casi nada. Solo es portable para Java, tiene problemas de seguridad, es dificil de mantener… Parece ser que en su lugar se usa JSON(thi King 👑), APIs REST (noidea), BDD y hasta XML.

CONCLUSIÓN: seguimos.

P.D: este es el tema que menos me ha gustado de todos con diferencia. 😅